CSV PROCESSING BENCHMARK

How fast and memory hungry are Ruby CSV options?

A reproducible benchmark comparing stdlib CSV, SmarterCSV, Polars, Polars low-memory, and DuckDB on generated person-account data.

Benchmark Setup

Dataset

- Rows generated with Faker.
- Columns: first name, last name, date of birth, account balance.
- Valid DOB format: `mm/dd/yyyy`.
- Invalid DOBs: roughly 10%, split between empty and `mm-dd-yy`.
- Balances range from -1000.00 to 1000000.00.

Measured Work

- Read the file.
- Count balances above 10000.00.
- Count duplicate rows.
- Total all balances.
- Count invalid DOB rows.
- Collect invalid DOB row indexes.
- Run all metrics together in one task.

Files And Stored Results

FILE	SIZE	COLUMNS	INVALID DOB TARGET
1K rows	34 KiB	first_name, last_name, date_of_birth, account_balance	10% invalid DOB
10K rows	340 KiB	first_name, last_name, date_of_birth, account_balance	10% invalid DOB
1M rows	33 MiB	first_name, last_name, date_of_birth, account_balance	10% invalid DOB

Charts in this deck use `benchmarks/results/all_implementations_all_task_validation.summary.json`. The stored data is a one-run validation summary, not the full repeated benchmark matrix.

Runner Model

1 task

Each task runs as its own Docker-measured process.

1 bundle

Each implementation activates only its own Gemfile.

2 memory points

Peak anonymous memory and post-load anonymous memory.

The `all` benchmark is also its own process. It is not a summary of the individual task runs.

A Minimal Task Implementation

This is the shape of the invalid DOB count task in the simplest Ruby CSV version.

```
require "csv"
require "date"

invalid = 0

CSV.foreach(file, headers: true).with_index do |row, index|
  dob = row["date_of_birth"].to_s
  match = /\A(\d{2})\/(\d{2})\/(\d{4})\z/.match(dob)
  valid = match && Date.valid_date?(
    match[3].to_i, match[1].to_i, match[2].to_i
  )

  invalid += 1 unless valid
end

puts invalid
```

Implementation Size

IMPLEMENTATION	LOC	FILES COUNTED	NOTES
Ruby CSV	197	implementations/ruby_csv_default.rb	stdlib CSV.foreach baseline
SmarterCSV	158	implementations/smarter_csv.rb	chunked rows from smarter_csv 1.16.3
Polars	121	implementations/polars.rb + implementations/polars_common.rb	polars-df 0.25.1 eager read
Polars low memory	121	implementations/polars_low_memory.rb + implementations/polars_common.rb	same Polars code path with low_memory: true
DuckDB	145	implementations/duckdb.rb	duckdb 1.5.2.0 Ruby binding

LOC excludes blank lines and full-line comments. Polars entries include the shared Polars implementation helper.

Quick Start: Ruby CSV

Dedicated minimal example, not benchmark code: read `people.csv` and count rows where `account_balance > 10000`.

```
require "csv"

high_balance_rows = 0

CSV.foreach("people.csv", headers: true) do |row|
  balance = row["account_balance"].to_f
  high_balance_rows += 1 if balance > 10_000
end

puts high_balance_rows
```

Quick Start: SmarterCSV

Dedicated minimal example, not benchmark code: read `people.csv` and count rows where `account_balance > 10000`.

```
require "smarter_csv"

csv_options = {
  chunk_size: 10_000, col_sep: ",", row_sep: "\n",
  headers: { only: :account_balance },
  convert_values_to_numeric: { only: :account_balance },
  quote_escaping: :double_quotes,
  strip_whitespace: false, verbose: :quiet
}

high_balance_rows = 0

SmarterCSV.process("people.csv", csv_options) do |rows|
  rows.each { |r| high_balance_rows += 1 if r[:account_balance] > 10_000 }
end

puts high_balance_rows
```

Quick Start: Polars

Dedicated minimal example, not benchmark code: read `people.csv` and count rows where `account_balance > 10000`.

```
require "polars-df"

df = Polars.read_csv(
  "people.csv",
  schema_overrides: { "account_balance" => Polars::Float64 }
)

result = df
  .select((Polars.col("account_balance") > 10_000).sum)
  .row(0)
  .first

puts result
```

Quick Start: Polars low memory

Dedicated minimal example, not benchmark code: read `people.csv` and count rows where `account_balance > 10000`.

```
require "polars-df"

df = Polars.read_csv(
  "people.csv",
  low_memory: true,
  schema_overrides: { "account_balance" => Polars::Float64 }
)

result = df
  .select((Polars.col("account_balance") > 10_000).sum)
  .row(0)
  .first

puts result
```

Quick Start: DuckDB

Dedicated minimal example, not benchmark code: read `people.csv` and count rows where `account_balance > 10000`.

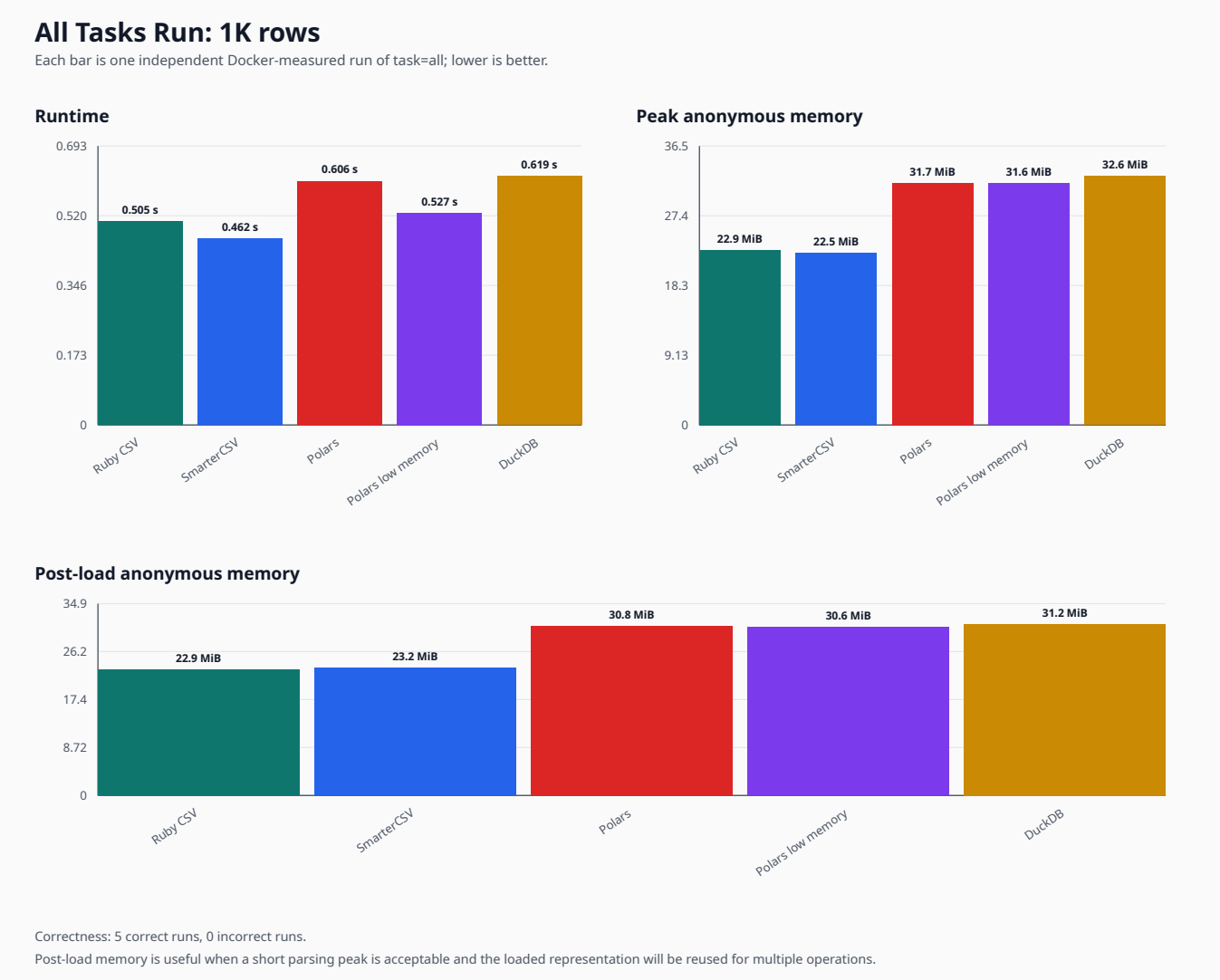
```
require "duckdb"

db = DuckDB::Database.open(":memory:")
con = db.connect

result = con.query(<<~SQL).first.first
  SELECT count(*)
  FROM read_csv('people.csv', header = true, auto_detect = true)
  WHERE account_balance > 10000
SQL

puts result
```

All Tasks: 1K Rows

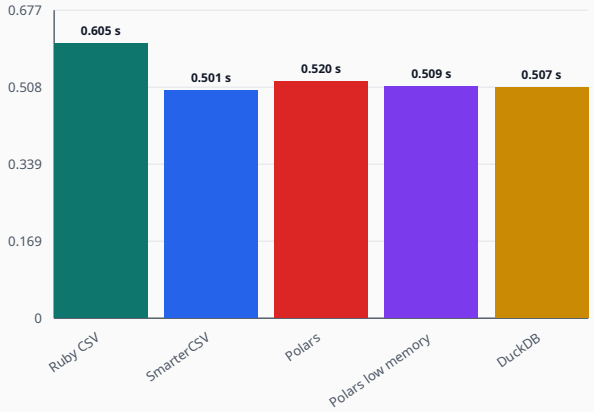


All Tasks: 10K Rows

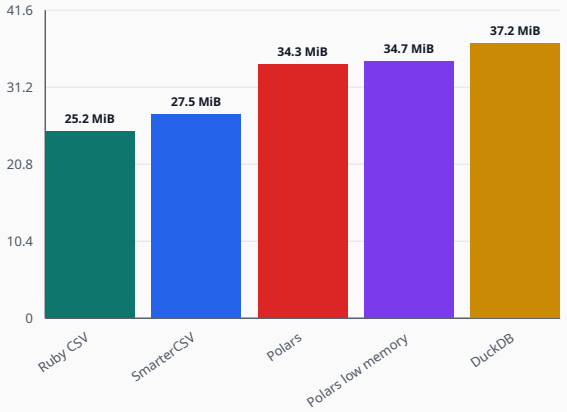
All Tasks Run: 10K rows

Each bar is one independent Docker-measured run of task=all; lower is better.

Runtime



Peak anonymous memory

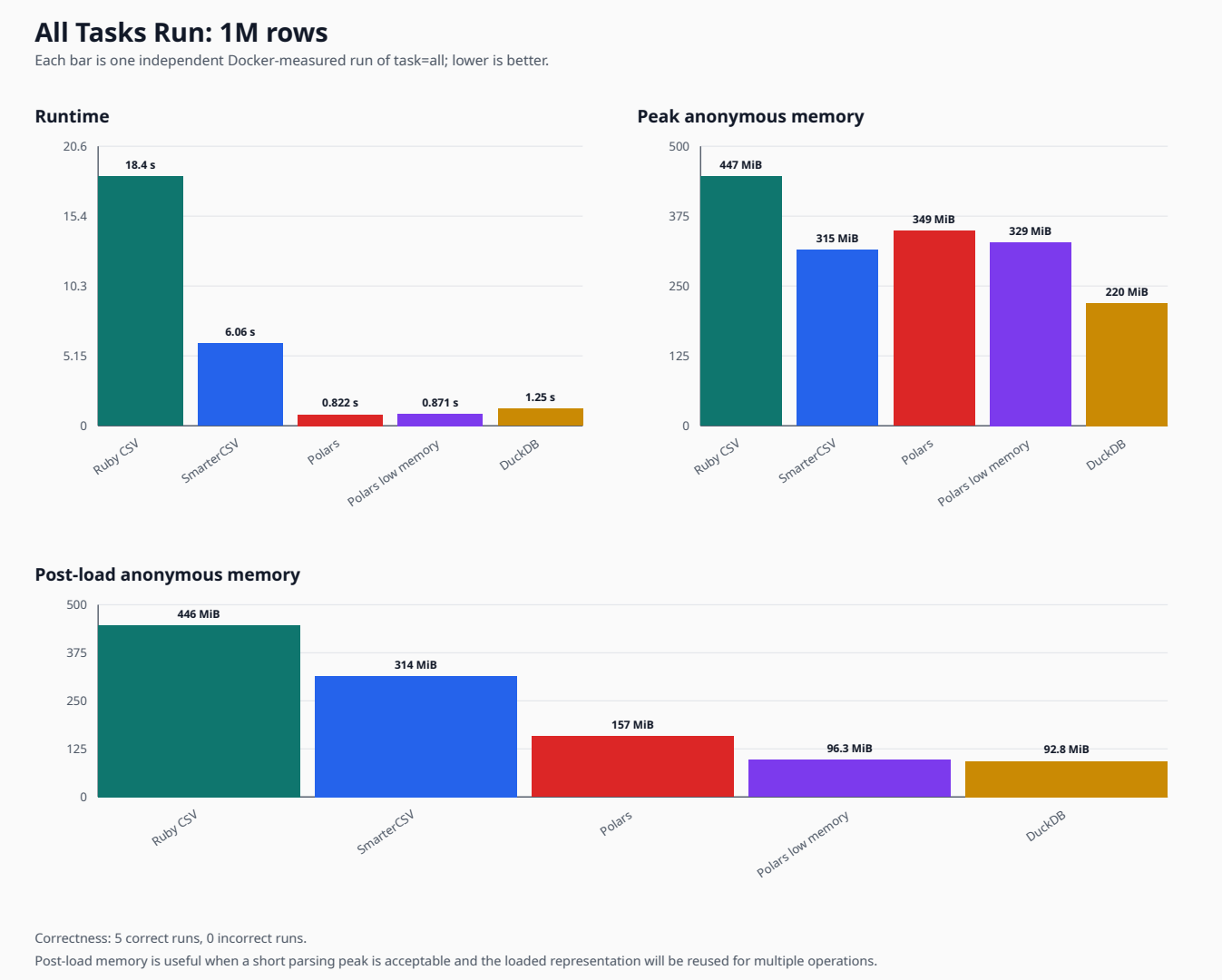


Post-load anonymous memory

Correctness: 5 correct runs, 0 incorrect runs.

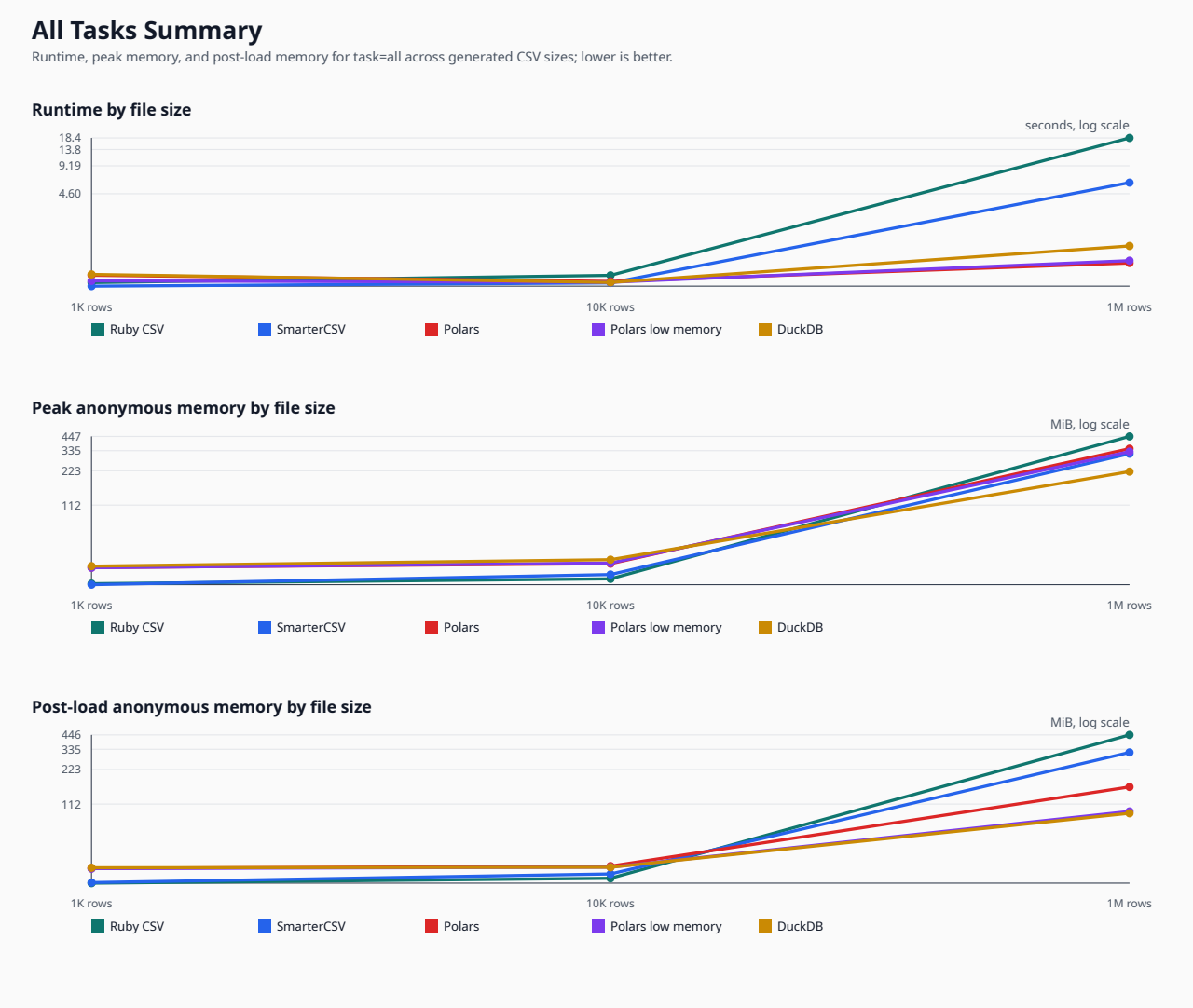
Post-load memory is useful when a short parsing peak is acceptable and the loaded representation will be reused for multiple operations.

All Tasks: 1M Rows



Post-Load Memory

Summary Across File Sizes

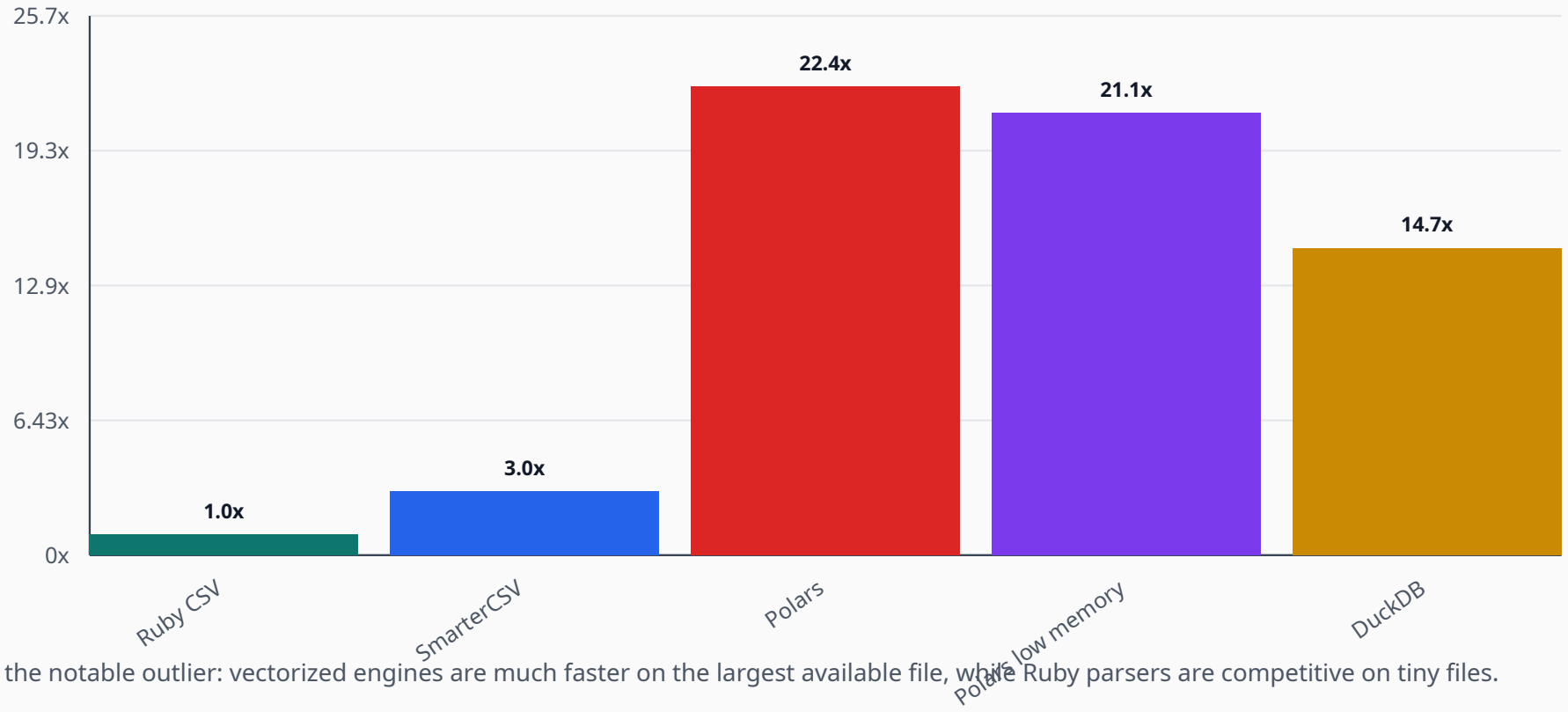


Surprise: Large-File Speedup

Notable Result: Large File Speedup

Derived from independent task=all runs; Ruby CSV is the 1.0x baseline.

Speedup vs Ruby CSV on 1M rows



This is the notable outlier: vectorized engines are much faster on the largest available file, while Ruby parsers are competitive on tiny files.

What Stands Out

Fastest 1M all-tasks run

Polars: 0.82 seconds.

Lowest 1M post-load memory

DuckDB: 92.8 MiB.

- Ruby CSV stays simple, but duplicate detection dominates memory on larger data.
- SmarterCSV improves the pure-Ruby path substantially for the combined workload.
- Polars and DuckDB pay more startup/setup cost on tiny files, then pull ahead on 1M rows.
- Post-load memory is the better metric when a transient parse peak is acceptable.

CONTRIBUTIONS WELCOME

Try it, break it, add another parser.

The benchmark runner is implementation-agnostic: add a script, add a manifest entry, and compare it against the same correctness checks.

https://github.com/kaiks/csv-benchmarking

- Add another Ruby CSV library.
- Improve the DuckDB or Polars implementations.
- Run the full repeated benchmark matrix on your machine.
- Send results, issues, and pull requests.